MoneyPoly: Code Quality Analysis & White-Box Testing Report

2024101063

March 24, 2026

Contents

1 Code Quality Analysis (Pylint)

1.1 Requirement coverage

This section addresses the requirement:
"Use pylint to analyze the codebase. Make sure to iteratively fix the code as per the warnings and suggestions. Document all the changes in the report. Save each iteration as a commit in the format `Iteration #: <What You Changed>`".

1.2 Project layout notes

During development, the project layout was reorganized. In particular, commit `10e4ebc` moved the runnable package under `whitebox/code/moneypoly` (and the entrypoint under `whitebox/code/main.py`).

1.3 How pylint was run

Because the importable package lives under `whitebox/code`, pylint was executed from that directory:

```
cd whitebox/code
python -m pylint moneypoly --score=y
```

1.4 Final pylint status

On the final state of the `main` branch, pylint reports:

- **Score:** 10.00/10

1.5 Iteration-by-iteration change log

Each pylint-driven cleanup pass was saved as an `Iteration #` commit. The table below documents **all iteration commits**, their hashes, and the **files touched** in each iteration. (The intent and file lists are taken directly from the commit history.)

Iter.	Hash	Commit subject	Files changed / what was updated
1	377b37e	Board Module fixed	Updated board implementation (`bo` to satisfy pylint and improve module
2	1ae7ab5	Config Module fixed	Updated configuration module (`cor` for lint compliance and clarity.
3	ed46260	Property module fixed	Updated `board.py` and `property.` dress lint findings across property/ teractions.
4	522ec59	UI module fixed	Updated UI helpers (`ui.py`) to satis and standardize formatting/inputs.
5	a570c26	Game module simple warnings fixed	Updated game loop (`game.py`); al duced `whitebox/bugs_fixed.md` a ning log.
6	fd81804	Cards module fixed and gitignore added	Updated cards/deck logic (`cards.py` `.gitignore`; removed committed ca facts.

Iter.	Hash	Commit subject	Files changed / what was updated
7	2778194	Dice module fixed + removed import error	Updated multiple (bank/board/dice/game/player) solve lint/import issues exposed changes.
8	590ffab	Bank module fixed	Updated `bank.py` and documentatio consistent validation and clean API b
9	d9ff76b	Player module basic warnings fixed	Updated `player.py` and documenta for cleaner state handling and lint ance.
10	e951079	Game module unused imports errors fixed	Updated `board.py` and `game.py` to unused imports and remaining lint sues.
11	10e4ebc	Final cleanup + refactor; moved code under whitebox	Refactored and reorganized: move age to `whitebox/code/moneypoly` trypoint to `whitebox/code/main.p`
12	057bc79	Move whitebox tests outside code folder	Moved tests to `whitebox/tests` an `conftest.py` to keep imports work

Note: The goal of the iteration commits is not feature work; each iteration is a focused lint-driven cleanup of the indicated module(s). The report records the exact commit boundaries so the grader can verify the iterative process.

2 White-Box Test Cases

2.1 Requirement coverage

This section addresses the requirement:
"Design white-box test cases based on the structure of the code. Your tests must cover: (i) All branches, (ii) Key variable states that could affect program behavior, (iii) Relevant edge cases. In the report, explain in simple words why each test case is needed, along with all the errors or logical issues found by the test."

2.2 Where the tests live (as per directory requirement)

Per the required directory structure, the white-box test suite is located at:

- `whitebox/tests`

2.3 How tests were run

Because the importable package lives under `whitebox/code`, the test runner is executed from `whitebox/` and `whitebox/tests/conftest.py` adds `whitebox/code` to the import path:

```
cd whitebox
python -m pytest -q tests
```

2.4 Test-case catalog (why each test is needed)

This subsection explains **each test case** in simple words: what branch/state/edge-case it targets and why that matters.

Test case	Why it is needed / what it covers (white-box intent)	Issue found
Bank (whitebox/tests/test_bank.py)		
`test_bank_initialization`	Verifies constructor initializes reserves and loan counters correctly; covers the default-state branch for an untouched Bank.	—
`test_bank_collect`	Ensures the collect path increases reserves; covers positive amount path.	—
`test_bank_pay_out`	Ensures payout decreases reserves and returns the paid amount; covers the normal successful payout branch.	—
`test_bank_pay_out_negative_or_zero`	Edge-case: payout should be a no-op for non-positive values to prevent silent balance corruption.	—
`test_bank_pay_out_insufficient_funds`	Branch: bank must refuse to overpay; validates exception path for insufficient reserves.	—
`test_bank_give_loan`	State change: issuing a loan must update player balance and bank loan accounting; also checks user-visible output.	—
`test_bank_give_loan_negative_or_zero`	Edge-case: loan requests of 0/negative should not create loan records.	—
`test_bank_summary_and_repr`	Sanity-checks reporting methods (summary/repr) so debugging output stays stable.	—
Board (whitebox/tests/test_board.py)		
`test_board_initialization`	Confirms the board is populated (properties and groups) and baseline tile typing works for position 0.	—
`test_get_tile_type`	Covers multiple decision paths in tile typing, including fallback for out-of-range positions (edge-case branch).	—
`test_get_property_at`	Covers property lookup returning a real Property vs `None` for non-property/out-of-bounds.	—
`test_is_purchasable`	Covers purchasability branch points: purchasable/unpurchasable due to non-property, ownership, and mortgaged state.	—
`test_is_special_tile`	Covers special-tile classification logic for multiple tiles and ensures non-special fallback behaves.	—
`test_ownership_queries`	Validates aggregation helpers (unowned/owned) so game logic can rely on them.	—
`test_board_repr`	Ensures representation string is stable for debugging and prints.	—
Cards (whitebox/tests/test_cards.py)		
`test_card_deck_initialization`	Confirms deck stores cards and computes cards-remaining correctly from initial state.	—
`test_card_deck_draw_and_peek`	Covers index/state transitions: peek must not advance, draw must advance, and exhaustion must trigger a reshuffle before the next cycle.	Error 9
`test_card_deck_empty`	Edge-case: empty deck must return `None` for draw/peek without crashing.	—
`test_card_deck_reshuffle`	Covers reshuffle branch: index resets and deck size preserved.	—
`test_card_deck_repr`	Ensures representation string includes card count and next index for debugging.	—
Dice (whitebox/tests/test_dice.py)		
`test_dice_initialization`	Confirms initial dice state and derived helpers; covers default values branch.	—
`test_dice_roll_bounds`	Edge-case and correctness: dice results must always remain within 1–6, total within 2–12.	Error 3
`test_dice_doubles_streak`	White-box branch: doubles streak increments on doubles and resets otherwise (patched random for determinism).	—

Test case	Why it is needed / what it covers (white-box intent)	Issue found
`test_dice_reset`	Ensures reset clears all state, important for multi-turn correctness.	—
`test_dice_describe`	Covers output formatting for both doubles and non-doubles branches.	—
`test_dice_repr`	Ensures repr remains stable for debugging.	—
Player (whitebox/tests/test_player.py)		
`test_player_initialization`	Verifies default player state (money, position, jail flags), which many game branches assume.	—
`test_player_initialization_custom_balance`	Covers the balance constructor path with non-default balance.	—
`test_add_money`	Ensures adding funds updates balance correctly.	—
`test_add_money_negative_raises_value_error`	Edge case: rejects negative additions to prevent silent balance corruption.	—
`test_deduct_money`	Ensures normal deduction decreases balance.	—
`test_deduct_money_negative_raises_value_error`	Edge case: rejects negative deductions (would be equivalent to adding money).	—
`test_is_bankrupt`	Covers the bankruptcy decision branch, including boundary at zero and negative balances.	—
`test_net_worth`	Confirms net worth equals balance when no properties are owned; baseline for winner logic.	—
`test_move_no_wrap`	Branch: moving without crossing GO must not award GO salary.	—
`test_move_with_wrap`	Branch: wrap-around must award GO salary and print message.	Error 1
`test_move_past_go`	Branch: crossing GO by more than board size must still award GO salary and compute final position.	Error 1
`test_go_to_jail`	State transition: go-to-jail must set jail flags and move to jail position.	—
`test_add_and_remove_property`	Covers property list manipulation including deduplication and removing missing entries.	—
`test_status_line`	Covers status-line formatting for jailed vs non-jailed (branch on jail state).	—
`test_repr`	Ensures repr is stable for debugging output.	—
`test_state_properties`	Covers setters/getters for key state vars (jail turns, jail cards, eliminated).	—
Property (whitebox/tests/test_property.py)		
`test_property_initialization`	Verifies default property state (owner/mortgage/houses/group) which affects rent logic branches.	—
`test_property_mortgage_value`	Checks mortgage-value calculation since it controls cashflow in mortgage branch.	—
`test_property_mortgage`	Covers mortgage branch and repeat-mortgage edge-case (must no-op).	—
`test_property_unmortgage`	Covers unmortgage branch, redemption cost, and repeated unmortgage edge-case.	—
`test_property_is_available`	Covers availability branches: unowned/owned/mortgaged.	—
`test_property_repr`	Ensures repr differs for unowned vs owned properties (branch).	—
`test_property_get_rent_base`	Covers base rent and mortgaged rent=0 branch.	—
`test_property_get_rent_full_group`	Branch: rent doubling requires all properties in a group owned by same player; regression to base rent if group breaks.	Error 2
`test_property_group_basic`	Covers group bookkeeping (size, add, prevent duplicates) relied upon by rent logic.	—
`test_property_group_all_owned_by`	Covers all-owned-by decision branch for various partial ownership states.	—

Test case	Why it is needed / what it covers (white-box intent)	Issue found
`test_property_group_owner_counts`	Covers aggregation of ownership counts per player, used for reporting/analysis.	—
UI (whitebox/tests/test_ui.py)		
`test_print_banner`	Smoke-test for banner output formatting; ensures consistent UX.	—
`test_print_player_card`	Covers printing when player is jailed and holds jail cards (state-driven branches).	—
`test_print_player_card_with_properties`	Covers printing when player owns properties and some are mortgaged.	—
`test_print_standings`	Covers ranking output and the jailed annotation branch.	—
`test_print_board_ownership`	Covers ownership table output and mortgaged marker.	—
`test_format_currency`	Ensures formatting helper works for large values.	—
`test_safe_int_input`	Edge-case: invalid/empty input returns default rather than crashing.	—
`test_confirm`	Covers yes/no parsing decision path; demonstrates that only 'y' is treated as yes.	—
Game (whitebox/tests/test_game.py)		
`test_game_initialization`	Verifies game bootstraps players, state, and turn counters correctly.	—
`test_advance_turn`	Covers round-robin turn advancement and turn-number increments (branch across wrap-around).	—
`test_find_winner`	Ensures winner selection uses maximum net worth (not minimum).	Error 4
`test_play_turn_jail`	Covers jailed-player branch where player chooses not to pay/escape; ensures jail turns increment.	—
`test_play_turn_doubles_to_jail`	Covers "three doubles" rule branch forcing jail.	—
`test_play_turn_invokes_interactive_menu`	Ensures the menu loop is integrated into normal turn flow in interactive runs.	Error 8
`test_buy_property_skip`	Covers unowned-property branch where user skips purchase; ensures no ownership changes.	—
`test_buy_property_buy`	Covers unowned-property branch where user buys; ensures balance/ownership update.	—
`test_buy_property_auction`	Covers unowned-property branch where user auctions; ensures bidding path assigns owner.	—
`test_auction_logic`	Covers auction branch where at least one bid wins; verifies affordability logic indirectly.	—
`test_auction_logic_all_pass`	Edge-case: auction with no bids must keep property unowned.	—
`test_pay_rent`	Covers rent-payment branch and validates cash transfer to the owner.	Error 6
`test_mortgage_and_unmortgage`	Covers mortgage/unmortgage flows and their state transitions on the property.	—
`test_trade_logic`	Covers trade path and ensures ownership and cash transfer are correct.	Error 7
`test_bankruptcy_skips`	Regression test ensuring bankrupt player removal does not skip the next player.	Error 5
`test_apply_card_actions`	Covers the card-action dispatch table and key state mutations (collect/pay/jail/jail_free/move/birthday).	—

3 Errors / Logical Issues Found and Fixed

The table below lists each discovered issue (via the tests above), the fixing commit, and the short explanation of what was wrong.

Err.	Hash	Primary test trigger	What was wrong (simple words) and what was changed
1	c374dc3	Player move wrap tests	Passing/landing on GO did not reliably award salary; fixed movement logic to add GO salary when wrapping.
2	d9ef01c	Property full-group rent test	Rent doubling logic for complete ownership sets was incorrect; fixed rent calculation to double only when group ownership condition holds.
3	9e52b9f	Dice roll bounds test	Dice roll range missed the value 6; fixed randomness bounds so dice results are 1–6.
4	ab3ac14	Winner selection test	Winner selection used the wrong comparator (min instead of max); fixed to return player with maximum net worth.
5	69ff35c	Bankruptcy skip regression	Removing a bankrupt player could mis-adjust `current_index` and skip the next player; fixed index update on elimination.
6	a7b0105	Rent payment test	Rent was not being transferred correctly to the property owner; fixed rent transfer to credit the owner.
7	3c28703	Trade test	Trade did not correctly transfer cash to the seller; fixed trade cashflow and ownership updates.
8	6d1706b	Pre-roll menu integration test	Mortgage/trade/menu logic existed but was unreachable from normal play; integrated menu into the turn flow for interactive runs.
9	a09756e	Card deck exhaustion test	Card decks cycled without reshuffling, making draws predictable after one full cycle; reshuffle now occurs automatically on exhaustion.